Introduction to Logic

Neuro-Symbolic Artificial Intelligence

Francesca Cairoli

University of Trieste, 2026/27

Let’s start with an example

Consider the interpersonal relations of a small group of friends with four members: *Abby*, *Bess*, *Cody*, and *Dana*.

Some of the girls like each other, but some do not.

Let's assume that we don’t know everything about the likes and dislikes of the girls.

By writing **logical sentences,** each informant can express exactly what he or she knows - no more, no less.

Dana likes Cody.	Direct sentence
*Abby does **not** like Dana.*	Tell us what is not true without saying what is true
*Dana does **not** like Abby.*	
*Bess likes Cody **or** Dana.*	Says that one condition holds or another but does not say which
*Abby likes **everyone** that Bess likes.*	General fact about the girls Abby likes
*Cody likes **everyone** who likes her.*	General fact about Cody's likes
***Nobody** likes herself*	Says something about everyone

	Abby	Bess	Cody	Dana
Abby			✓	
Bess			✓	
Cody	✓	✓		✓
Dana			✓	

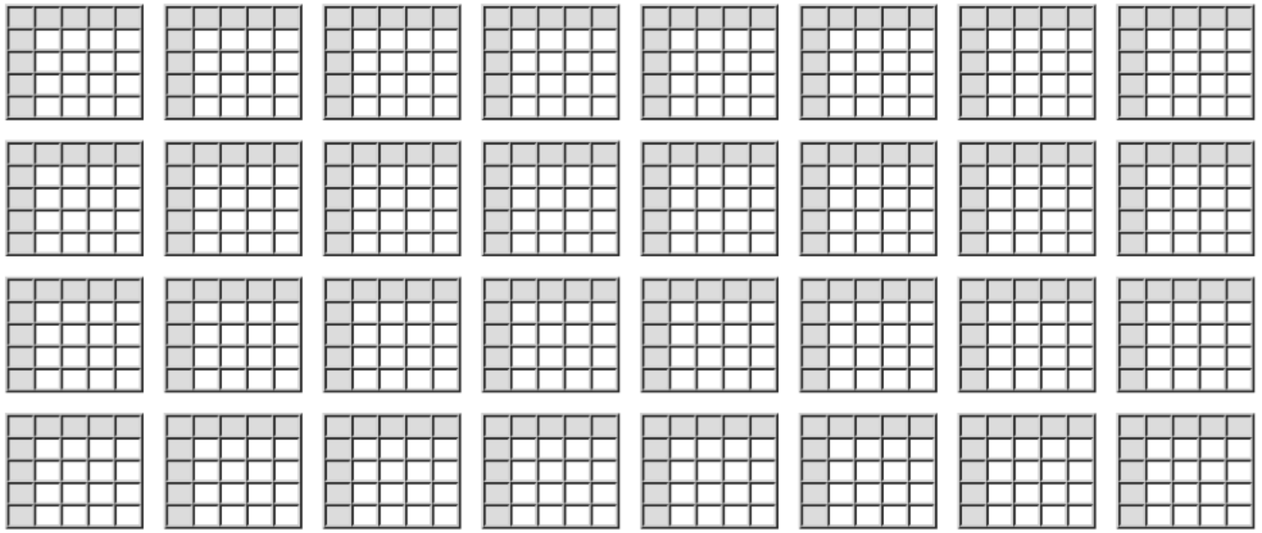
	Abby	Bess	Cody	Dana
Abby	✓		✓	
Bess		✓		✓
Cody	✓		✓	
Dana		✓		✓

Two possible **worlds**

These sentences are true in the first world but some are false in the second world. Hence, we can rule out the second world.

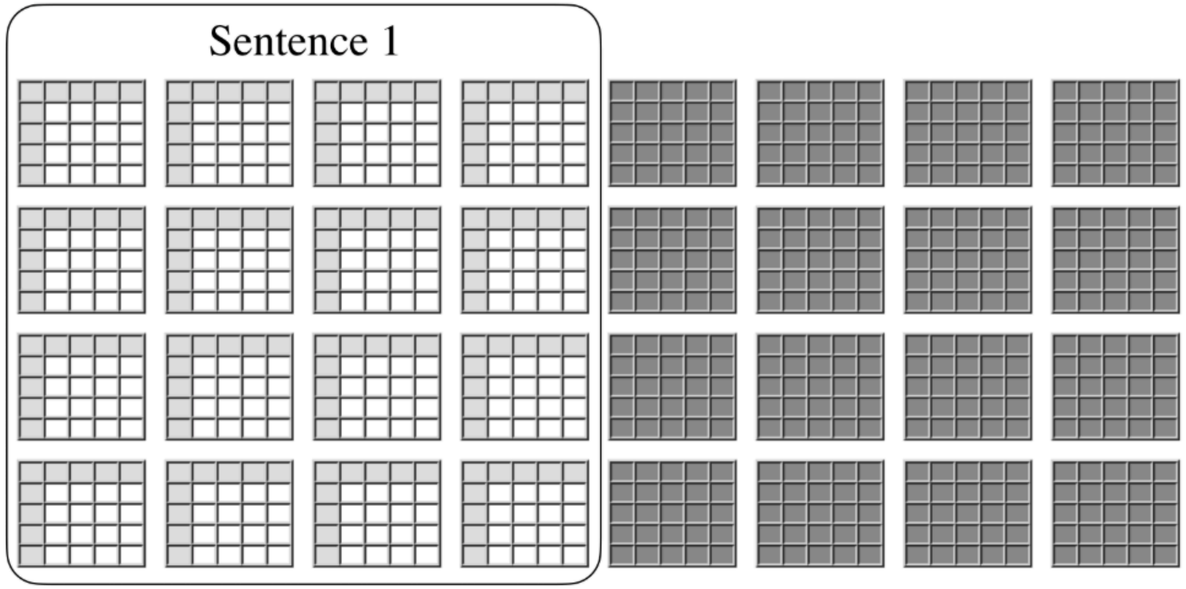
Possible worlds

In general, there are more than two **possible worlds** to consider (given four girls, there are sixteen possible instances of the *likes* relation). Each of these sixteen can be either true or false. There are 2^{16} (65,536) possible combinations of these true-false possibilities, and so there are 2^{16} possible worlds.



Possible worlds and logical sentences

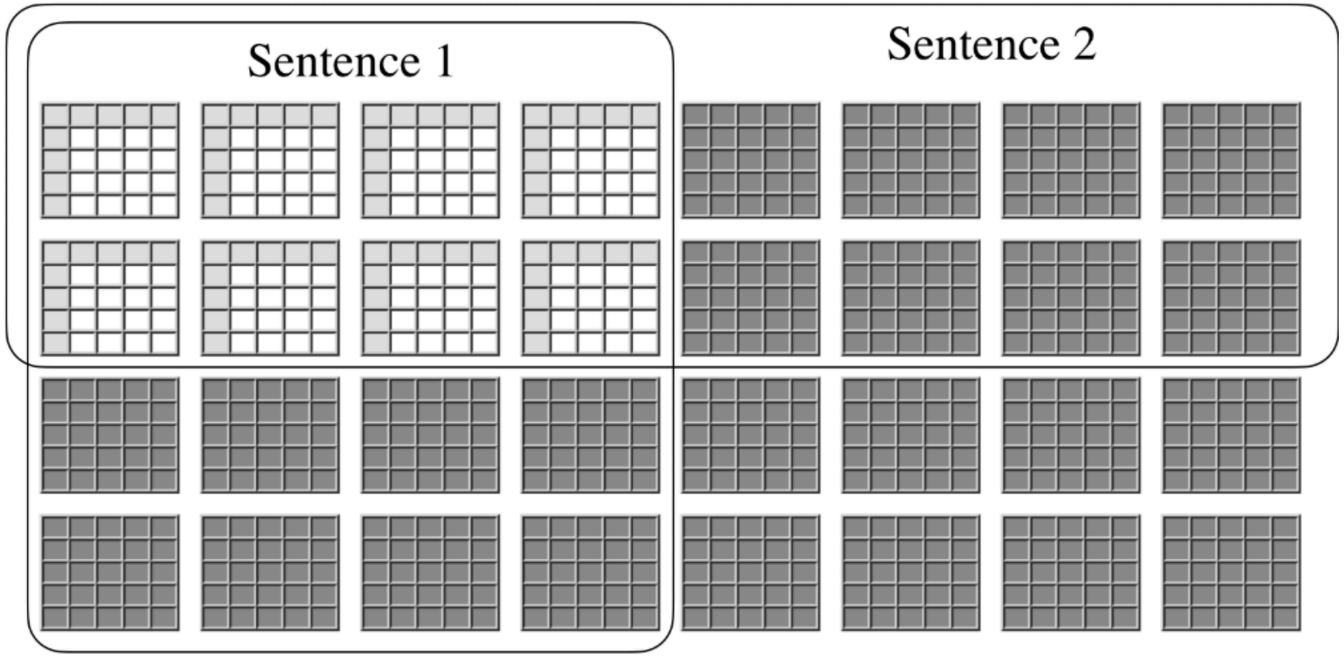
In general, there are more than two possible worlds to consider (given four girls, there are sixteen possible instances of the *likes* relation). Each of these sixteen can be either true or false. There are 2^{16} (65,536) possible combinations of these true-false possibilities, and so there are 2^{16} possible worlds.



Logical sentences constrain the possible ways the world could be. Each sentence divides the set of possible worlds into two subsets, those in which the sentence is true and those in which the sentence is false

Possible worlds and logical sentences

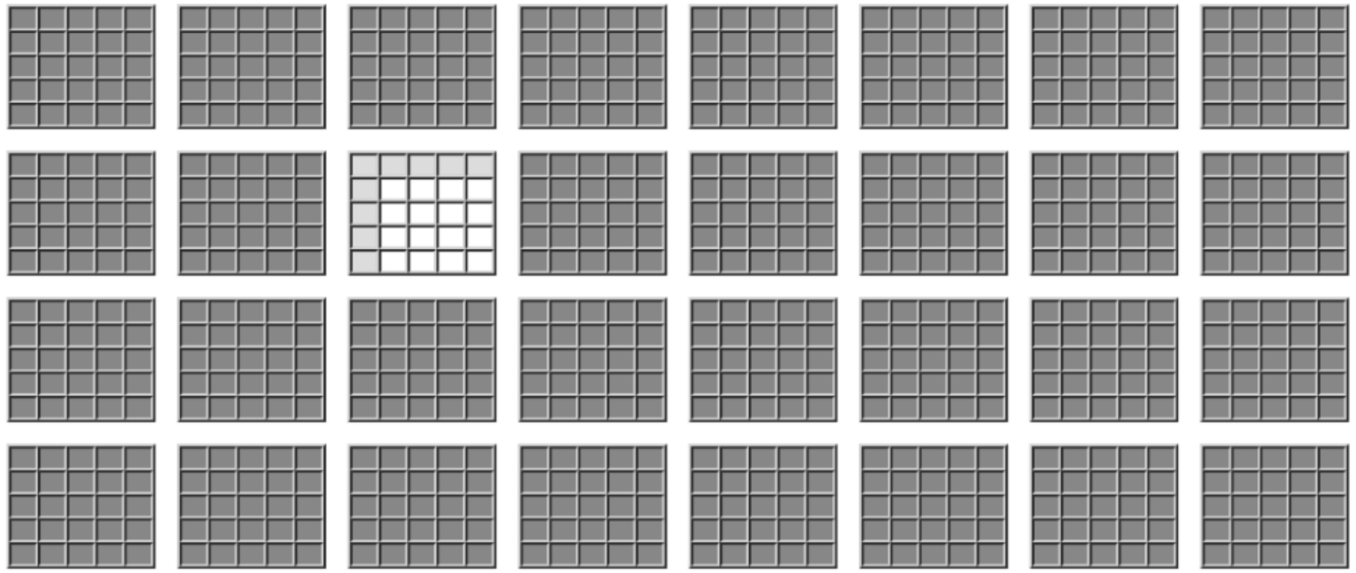
In general, there are more than two possible worlds to consider (given four girls, there are sixteen possible instances of the *likes* relation). Each of these sixteen can be either true or false. There are 2^{16} (65,536) possible combinations of these true-false possibilities, and so there are 2^{16} possible worlds.



Given two sentences, we know the world must be in the **intersection of the set of worlds in which the first sentence is true and the set of worlds in which the second sentence is true**.

Possible worlds and logical sentences

In general, there are more than two possible worlds to consider (given four girls, there are sixteen possible instances of the *likes* relation). Each of these sixteen can be either true or false. There are 2^{16} (65,536) possible combinations of these true-false possibilities, and so there are 2^{16} possible worlds.



Ideally, when we have enough sentences, we know exactly how things stand.

Logic Entailment

Although logical sentences can sometimes pinpoint a specific world among many possible worlds, this is not always the case. Sometimes, a collection of sentences only partially constrains the world. For example, there are four different worlds that satisfy the sentences in the previous section.

	Abby	Bess	Cody	Dana
Abby			✓	
Bess			✓	
Cody	✓	✓		✓
Dana			✓	

	Abby	Bess	Cody	Dana
Abby		✓	✓	
Bess			✓	
Cody	✓	✓		✓
Dana			✓	

	Abby	Bess	Cody	Dana
Abby			✓	
Bess			✓	
Cody	✓	✓		✓
Dana		✓	✓	

	Abby	Bess	Cody	Dana
Abby		✓	✓	
Bess			✓	
Cody	✓	✓		✓
Dana		✓	✓	

In situations like this, **which world should we use in answering questions?**

The good news it that sometimes it does not matter. Even though a set of sentences does not determine a unique world, there are some sentences that have the same truth value in *every* world that satisfies the given sentences, and we can use that value in answering questions.

This is **logical entailment**. We say that a set of premises *logically entails* a conclusion if and only if **every world that satisfies the premises also satisfies the conclusion**.

Logic Entailment

	Abby	Bess	Cody	Dana
Abby			✓	
Bess			✓	
Cody	✓	✓		✓
Dana			✓	

	Abby	Bess	Cody	Dana
Abby		✓	✓	
Bess			✓	
Cody	✓	✓		✓
Dana			✓	

	Abby	Bess	Cody	Dana
Abby			✓	
Bess			✓	
Cody	✓	✓		✓
Dana		✓	✓	

	Abby	Bess	Cody	Dana
Abby		✓	✓	
Bess			✓	
Cody	✓	✓		✓
Dana		✓	✓	

What can we conclude from the bits of information in our sample logical sentences? Quite a bit, as it turns out.

- In our example, we see that Bess likes Cody in all four worlds.
- We also see Bess does *not* like Abby in all four worlds.
- Does Dana like Bess? Since there are different values in different worlds, we cannot say yes and we cannot say no. All we can say is Maybe. In the real world, either Dana likes Bess or she doesn't. However, we do not have enough information to say which case is correct.

Model Checking

	Abby	Bess	Cody	Dana
Abby			✓	
Bess			✓	
Cody	✓	✓		✓
Dana			✓	

	Abby	Bess	Cody	Dana
Abby		✓	✓	
Bess			✓	
Cody	✓	✓		✓
Dana			✓	

	Abby	Bess	Cody	Dana
Abby			✓	
Bess			✓	
Cody	✓	✓		✓
Dana		✓	✓	

	Abby	Bess	Cody	Dana
Abby		✓	✓	
Bess			✓	
Cody	✓	✓		✓
Dana		✓	✓	

Model checking is the process of examining the set of all worlds to determine logical entailment. To check whether a set of sentences logically entails a conclusion, we use our premises to determine which worlds are possible and then examine those worlds to see whether or not they satisfy our conclusion. If the number of worlds is not too large, this method works well.

Model Checking



	Abby	Bess	Cody	Dana
Abby			✓	
Bess			✓	
Cody	✓	✓		✓
Dana			✓	

	Abby	Bess	Cody	Dana
Abby		✓	✓	
Bess			✓	
Cody	✓	✓		✓
Dana			✓	

	Abby	Bess	Cody	Dana
Abby			✓	
Bess			✓	
Cody	✓	✓		✓
Dana		✓	✓	

	Abby	Bess	Cody	Dana
Abby		✓	✓	
Bess			✓	
Cody	✓	✓		✓
Dana		✓	✓	

Model checking is the process of examining the set of all worlds to determine logical entailment. To check whether a set of sentences logically entails a conclusion, we use our premises to determine which worlds are possible and then examine those worlds to see whether or not they satisfy our conclusion. If the number of worlds is not too large, this method works well.

Unfortunately, in general, there are many, many possible worlds; and, in some cases, the number of possible worlds is infinite, in which case model checking is **impossible**. So what do we do? The answer is logical reasoning and logical proofs.

Rules of Inference

Logical proofs are analogous to derivations in algebra. We can try solving algebraic equations by randomly trying different values for the variables in those equations. However, we can usually get to an answer faster by manipulating our equations syntactically.

Logical reasoning is similar. Rather than checking all worlds, we simply apply syntactic operations to the premises we are given to generate conclusions.

Such operations are typically called **rules of inference**. By applying rules of inference to premises, we produce conclusions that are entailed by those premises. A **proof** is a sequence of such rule applications.

Example: All Accords are Hondas.

All Hondas are Japanese.

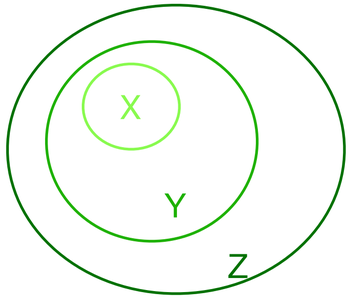
Therefore, all Accords are Japanese.

All x are y.

All y are z.

Therefore, all x are z.

Rules of Inference



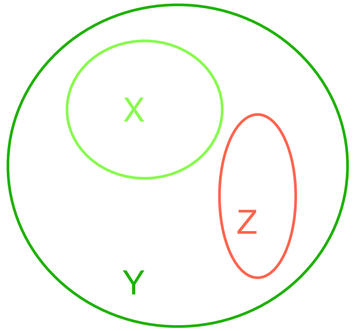
All x are y.

All y are z.

Therefore, all x are z.

Correctness in logical reasoning is determined by the logical operators in our sentences, not the objects and relationships mentioned in those sentences. Second, the conclusion is guaranteed to be true only if the premises are true.

There are patterns that are just plain wrong in the sense that they can lead to incorrect conclusions.



All Toyotas are cars.

Some cars are Porsches.

Therefore, some Toyotas are Porsches.

All x are y.

Some y are z.

Therefore, some x are z.

Deduction

What distinguishes a correct pattern from one that is incorrect is that it must *always* lead to correct conclusions, i.e. they must be correct so long as the premises on which they are based are correct. As we will see, this is the defining criterion for what we call **deduction***.*

Now, it is noteworthy that there are patterns of reasoning that are not always correct but are sometimes useful. There is induction, abduction, analogy, and so forth.

Induction

Induction is reasoning from the particular to the general.

Although we have seen many ravens, we have not seen them all. However, if we see enough cases in which something is true and we never see a case in which it is false, we tend to conclude that it is always true. Unfortunately, when induction is incomplete, as in this case, it is not sound. There might be an albino raven that we have not yet seen.

> *I have seen 1000 black ravens.*
>
> *I have never seen a raven that is not black.*
>
> *Therefore, every raven is black.*

Incomplete induction is the basis for science and machine learning. Science aspires to discover / propose new knowledge.

Deduction is the subject matter of Logic. Logic aspires to apply and/or analyze existing knowledge.

Logical entailment and provability

A set of premises **logically entails** a conclusion if and only if every possible world that satisfies the premises also satisfies the conclusion.

A sentence is **provable** from a set of premises if and only if there is a finite sequence of sentences in which every element is either a premise or the result of applying a deductive rule of inference to earlier members in the sequence.

These concepts are quite **different**. One is based on possible worlds; the other is based on symbolic manipulation of expressions.

Yet, for "well-behaved" logics, it turns out that logical entailment and provability are identical - a set of premises logically entails a conclusion if and only if the conclusion is provable from the premises. Even if the number of worlds is *infinite*, it is possible in such logics to produce a *finite* proof of the conclusion, i.e. we can determine logical entailment without going through all possible worlds.

Rules of Inference

Dana likes Cody.

*Abby does **not** like Dana.*

*Dana does **not** like Abby.*

*Bess likes Cody **or** Dana.*

*Abby likes **everyone** that Bess likes.*

*Cody likes **everyone** who likes her.*

***Nobody** likes herself*

```
Rel(D,C)

¬ Rel(A, D)

¬ Rel(D, A)

Rel(B, C) ∨ Rel(B, D)

∀X: Rel(B,X) → Rel(A,X)

∀X: Rel(X,C) → Rel(C,X)

∀X: ¬Rel(X,X)
```

logic inference →

```
Rel(B,C)
```

logic inference →

```
¬ Rel(B,A)
```

Inference rules used: Universal Instantiation, Modus Tollens, and Disjunctive Syllogism.

Prolog queries:

`00_likes.pl`

ASP proofs:

`00_likes_proof.lp`

Propositional Logic

Propositional Logic

The language of propositional logic is based on **propositions**:

declarative sentences which one can, in principle, argue as being "true" or "false".

Propositional symbols are the **atomic formulas** of the language.

More complex sentences are constructed using **logical connectives** (AND, OR, NOT, IF…THEN, etc.).

In classical propositional logic (PL) each sentence is either true or false. In fact, the content of the propositions is not relevant to PL. PL is not the study of truth, but of the relationship between the truth of one statement and that of another.

Propositional Logic

Examples of propositions:

Statement	Truth Value
The sun rises in the East and sets in the West.	TRUE
1 + 1 = 2	TRUE
‘b’ is a vowel.	FALSE

Connective Simbol	Term	Word	Example
$\wedge$	Conjunction	AND	$A \wedge B$
$\vee$	Disjunction	OR	$A \vee B$
$\neg$	Negation	NOT	$\neg A$
$\Rightarrow$	Implication	IMPLIES	$A \Rightarrow B$
$\Leftrightarrow$	Biconditional	IF AND ONLY IF	$A \Leftrightarrow B$

Propositional Logic

Examples of propositions:

Statement	Truth Value
The sun rises in the East and sets in the West.	TRUE
1 + 1 = 2	TRUE
‘b’ is a vowel.	FALSE

Examples of non-propositions:

Some sentences are not propositions because they don’t have a definite truth value or may vary depending on context:

- “What time is it?” (Question)
- “Go out and play.” (Command)
- “x + 1 = 2” (Open sentence — depends on the value of x)

Connective Simbol	Term	Word	Example
$\wedge$	Conjunction	AND	$A \wedge B$
$\vee$	Disjunction	OR	$A \vee B$
$\neg$	Negation	NOT	$\neg A$
$\Rightarrow$	Implication	IMPLIES	$A \Rightarrow B$
$\Leftrightarrow$	Biconditional	IF AND ONLY IF	$A \Leftrightarrow B$

Truth tables of PL

To know the truth value of a proposition in all possible scenarios, we consider all the possible combinations of the propositions which are joined together by logical connectives to form the given compound proposition. This compilation of all possible scenarios in a tabular format is called a **truth table.**

Boolean operators and truth tables ($0 = false, 1 = true$)

x_1	x_2	$x_1 \wedge x_2$
0	0	0
0	1	0
1	0	0
1	1	1

$val(x_1 \wedge x_2) = \min(x_1, x_2)$

x_1	x_2	$x_1 \vee x_2$
0	0	0
0	1	1
1	0	1
1	1	1

$val(x_1 \vee x_2) = \max(x_1, x_2)$

x_1	$\neg x_1$
0	1
1	0

$val(\neg x_1) = 1 - val(x_1)$

x_1	x_2	$x_1 \Rightarrow x_2$
0	0	1
0	1	1
1	0	0
1	1	1

$val(x_1 \Rightarrow x_2) = 1$
iff $val(x_1) \leq val(x_2)$

x_1	x_2	$x_1 \Leftrightarrow x_2$
0	0	1
0	1	0
1	0	0
1	1	1

$val(x_1 \Leftrightarrow x_2) = 1$
iff $val(x_1) = val(x_2)$

Boolean Logic

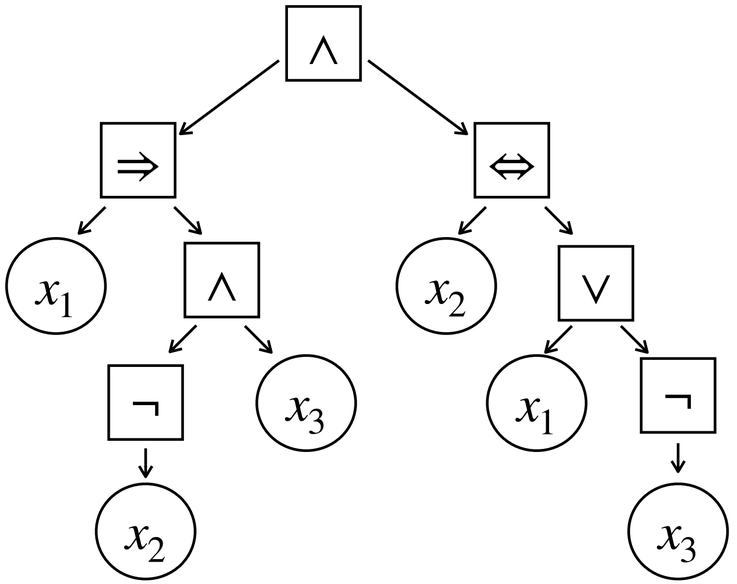
Boolean logic formula takes a set of I variables $\{x_i\}_{i=1}^{I} \in \{false, true\}$ and combines them using Boolean operators, returning $true$ or $false$.

For example: $\phi := (x_1 \Rightarrow (\neg x_2 \wedge x_3)) \wedge (x_2 \Leftrightarrow (\neg x_3 \vee x_1))$

Boolean Logic

Boolean logic formula takes a set of I variables $\{x_i\}_{i=1}^{I} \in \{false, true\}$ and combines them using Boolean operators, returning $true$ or $false$.

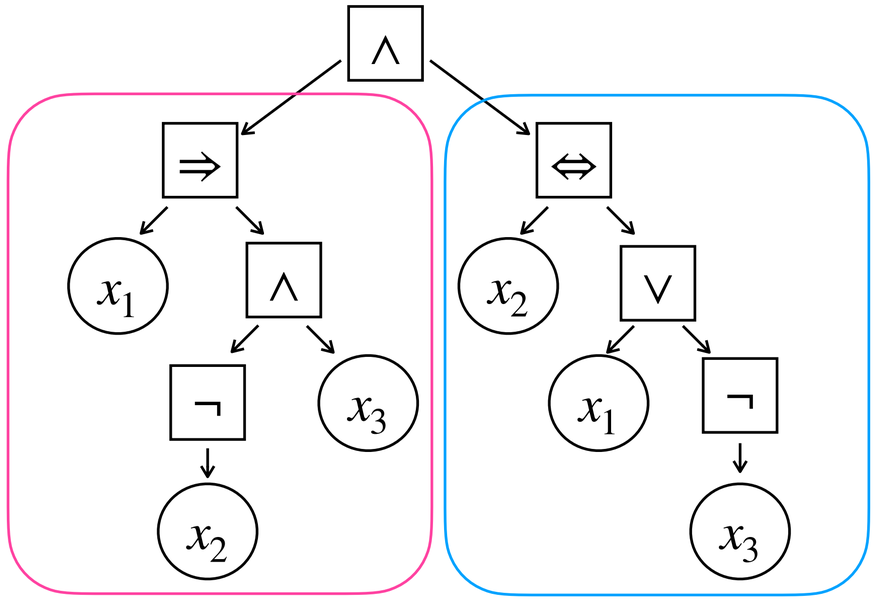
For example: $\phi := (x_1 \Rightarrow (\neg x_2 \wedge x_3)) \wedge (x_2 \Leftrightarrow (\neg x_3 \vee x_1))$



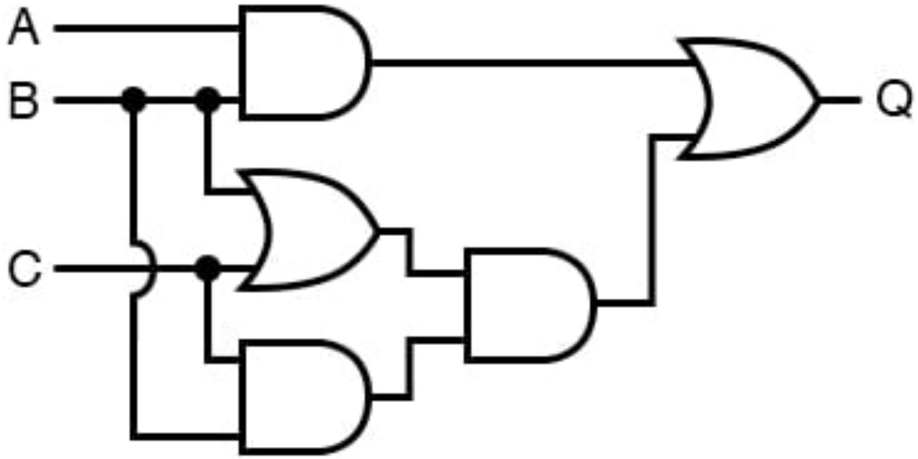
Boolean Logic

Boolean logic formula takes a set of I variables $\{x_i\}_{i=1}^{I} \in \{false, true\}$ and combines them using Boolean operators, returning $true$ or $false$.

For example: $\phi := (x_1 \Rightarrow (\neg x_2 \wedge x_3)) \wedge (x_2 \Leftrightarrow (\neg x_3 \vee x_1))$



Logic circuits



x_1	x_2	$x_1 \wedge x_2$
0	0	0
0	1	0
1	0	0
1	1	1

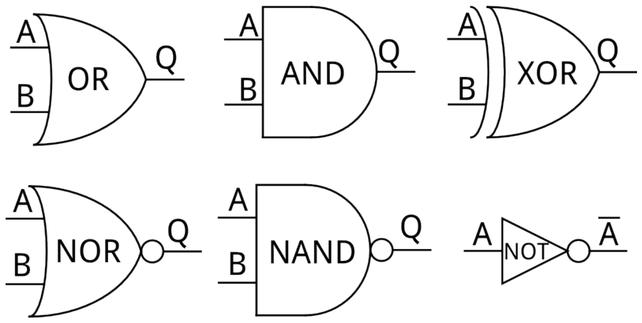
$val(x_1 \wedge x_2) = \min(x_1, x_2)$

x_1	x_2	$x_1 \vee x_2$
0	0	0
0	1	1
1	0	1
1	1	1

$val(x_1 \vee x_2) = \max(x_1, x_2)$

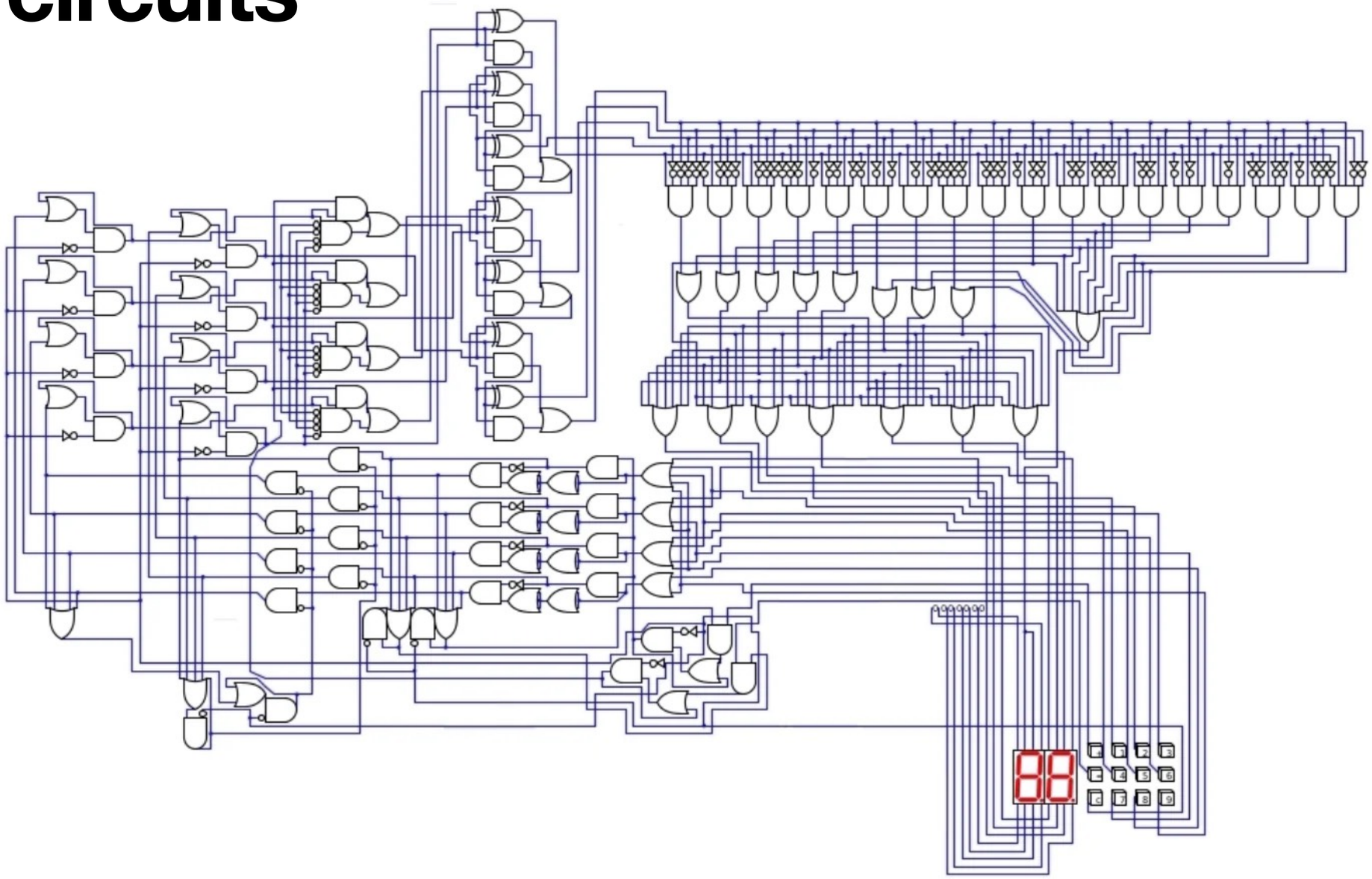
x_1	$\neg x_1$
0	1
1	0

$val(\neg x_1) = -\,val(x_1)$



$$Q = (A \wedge B) \vee \Big((B \vee C) \wedge (B \wedge C)\Big)$$

Logic circuits



Tautologies

Two formulas are **logically equivalent** if the truth outputs are the same.

p	q	$p \Rightarrow p$	$\neg p$	$p \vee \neg p$	$p \wedge \neg p$	$\neg(p \wedge \neg p)$
1	1	1	0	1	0	1
1	0	1	0	1	0	1
0	1	1	1	1	0	1
0	0	1	1	1	0	1

Conditionals

Conditionals can be rewritten as three other forms.

Conditional: $p \Rightarrow q$

Inverse: $\neg p \Rightarrow \neg q$ (negate)

Converse: $q \Rightarrow p$ (swap)

Contrappositive: $\neg q \Rightarrow \neg p$ (negate + swap)

Conditionals

Conditionals can be rewritten as three other forms.

Conditional: $p \Rightarrow q$

Inverse: $\neg p \Rightarrow \neg q$ (negate)

Converse: $q \Rightarrow p$ (swap)

Contrappositive: $\neg q \Rightarrow \neg p$ (negate + swap)

p	q	$p \Rightarrow q$	$\neg q \Rightarrow \neg p$	$q \Rightarrow p$	$\neg p \Rightarrow \neg q$
1	1	1	1	1	1
1	0	0	0	1	1
0	1	1	1	0	0
0	0	1	1	0	0

Laws of logic

We can substitute *equivalent formulas* using the laws of logic.

DeMorgan's Laws:

$$\neg(p \wedge q) \Leftrightarrow \neg p \vee \neg q$$

$$\neg(p \vee q) \Leftrightarrow \neg p \wedge \neg q$$

Distributive Laws:

$$p \wedge (q \vee r) \Leftrightarrow (p \wedge q) \vee (p \wedge r)$$

$$p \vee (q \wedge r) \Leftrightarrow (p \vee q) \wedge (p \vee r)$$

Contrappositive Law:

$$p \Rightarrow q \Leftrightarrow \neg q \Rightarrow \neg p$$

Conditional Law:

$$p \Rightarrow q \Leftrightarrow \neg p \vee q$$

Associative Law:

$$p \wedge (q \wedge r) \Leftrightarrow (p \wedge q) \wedge r$$

$$p \vee (q \vee r) \Leftrightarrow (p \vee q) \vee r$$

PL Example

Consider the following statements:

- P: Good mobile phones are not cheap.
- Q: Cheap mobile phones are not good.
- L: P implies Q
- M: Q implies P
- N: P is equivalent to Q

Which one of the following about L, M, and N is CORRECT?

(A) Only L is TRUE.

(B) Only M is TRUE.

(C) Only N is TRUE.

(D) L, M and N are TRUE.

PL Example

Consider the following statements:

- P: Good mobile phones are not cheap.
- Q: Cheap mobile phones are not good.
- L: P implies Q
- M: Q implies P
- N: P is equivalent to Q

Which one of the following about L, M, and N is CORRECT?

(A) Only L is TRUE.

(B) Only M is TRUE.

(C) Only N is TRUE.

(D) L, M and N are TRUE.

Solution

Let a and b be two proposition
a: Good Mobile phones.
b: Cheap Mobile Phones.

P and Q can be written in logic as
P: $a \Rightarrow \neg b$
Q: $b \Rightarrow \neg a$.

Truth Table

a	b	$\neg a$	$\neg b$	P	Q
T	T	F	F	F	F
T	F	F	T	T	T
F	T	T	F	T	T
F	F	T	T	T	T

it clearly shows P and Q are equivalent.
so option D is Correct

PL Example

Consider the following statements:

- P: Good mobile phones are not cheap.
- Q: Cheap mobile phones are not good.
- L: P implies Q
- M: Q implies P
- N: P is equivalent to Q

Which one of the following about L, M, and N is CORRECT?

(A) Only L is TRUE.

(B) Only M is TRUE.

(C) Only N is TRUE.

(D) L, M and N are TRUE.

Exercise: Which one of the following is not equivalent to $p \Leftrightarrow q$:

(A) $(\neg p \vee q) \wedge (p \vee \neg q)$

(B) $(\neg p \vee q) \wedge (q \Rightarrow p)$

(C) $(\neg p \wedge q) \vee (p \wedge \neg q)$

(D) $(\neg p \wedge \neg q) \vee (p \wedge q)$

Solution

Let a and b be two proposition
a: Good Mobile phones.
b: Cheap Mobile Phones.

P and Q can be written in logic as
P: $a \Rightarrow \neg b$
Q: $b \Rightarrow \neg a$.

Truth Table

a	b	$\neg a$	$\neg b$	P	Q
T	T	F	F	F	F
T	F	F	T	T	T
F	T	T	F	T	T
F	F	T	T	T	T

it clearly shows P and Q are equivalent.
so option D is Correct

Logical Inference

Allows us to get from premises to a conclusion deductively

Modus ponens:

Premises: $p; p \Rightarrow q$

Conclusion: q

Example: **If** you eat your dinner **then** you get dessert. You ate dinner. So you get dessert.

Modus Tollens:

Premises: $\neg q; p \Rightarrow q$

Conclusion: $\neg p$

Example: **If** it rained last night, **then** my car will be wet. My car is **not** wet. So it did **not** rain last night.

Disjuctive Syllogism:

Premises: $p \vee r; \neg r$

Conclusion: p

Example: **Either** my keys are in my purse, **or** they are on the kitchen counter. You look in your purse thoroughly. The keys are **not** in there. Therefore, my keys must be on the kitchen counter.

Logical Inference

Allows us to get from premises to a conclusion deductively

Conjuction Introduction:

Premises: p; q

Conclusion: $p \wedge q$

Conjuction Elimination:

Premises: $p \wedge q$

Conclusion: p; q

Disjunction Introduction:

Premises: p

Conclusion: $p \vee q$

Double Negation:

$$\neg \neg p \Leftrightarrow p$$

Contraposition:

$$p \Rightarrow q \Leftrightarrow \neg q \Rightarrow \neg p$$

Logical Inference Exercise

Assumptions:

1. $p \wedge q$ is true
2. $q \Rightarrow s$ is true
3. $\neg r \vee \neg s$ is true

   $\rightarrow$ truth of $\neg r$??

Inference:

a. q is true for *Conjunction Elimination*

b. s is true for *Modus Ponens*

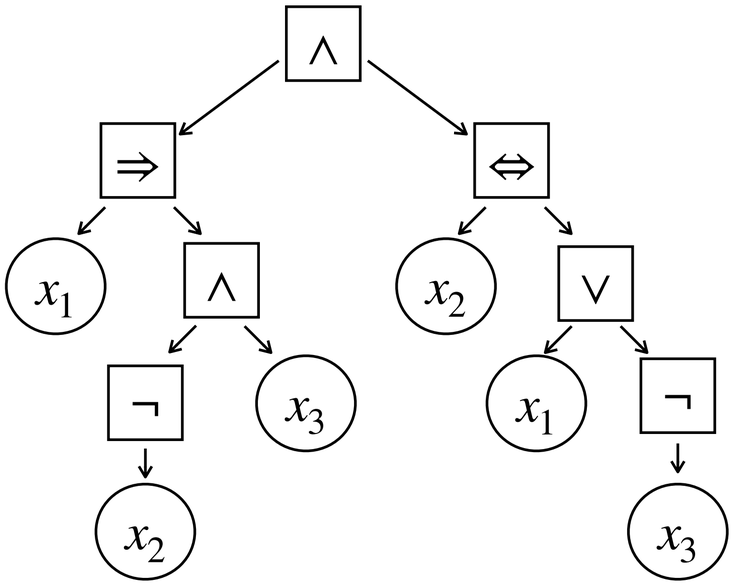
c. $\neg r$ is true for the *Disjunction Syllogism*

SAT Solvers

SAT Problem

Boolean Satisfiability

The *Boolean satisfiability problem* asks whether there is *at least one* combination of binary input variables $x_i \in \{false, true\}$, for which a Boolean logic formula returns $true$. When this is the case, we say the formula is ***satisfiable***.



A **SAT solver** is an algorithm for *establishing satisfiability*.

It takes the Boolean logic formula as input and returns **SAT** if it finds a combination of variables that can satisfy it or **UNSAT** if it can demonstrate that no such combination exists.

In addition, it may sometimes return **without an answer** if it cannot determine whether the problem is SAT or UNSAT.

Conjuntive normal form

To solve the SAT problem, we first convert the Boolean logic formula to a standard form that it is more amenable to algorithmic manipulation. Any formula can be re-written as a conjunction of disjunctions (i.e., the logical AND of statements containing OR relations).

$$\phi := (x_1 \vee x_2 \vee x_3) \wedge (\neg x_1 \vee x_2 \vee x_3) \wedge (x_1 \vee x_2 \vee \neg x_3)$$

Each term in brackets is known as a **clause** and combines together variables and their complements with a series of logical ORs. The clauses themselves are combined via AND relations.

Certificate

When a solver returns SAT or UNSAT, it also returns a **certificate**, which can be used to check the result with a simpler algorithm.

If the solver returns SAT, then the certificate will be a set of variables that obey the formula. These can obviously be checked by simply computing the formula with them and checking that it returns $true$.

If it returns UNSAT then the certificate will usually be a complex data structure that depends on the solver.

SAT problem is NP-complete

The SAT problem is proven to be **NP-complete** (NP+ NP-hard) and it follows that there is no known polynomial algorithm for establishing satisfiability in the general case. An important exception to this statement is 2-SAT for which a polynomial algorithm is known. However, for 3-SAT and above the problem is very difficult.

The **good news** is that modern SAT solvers are very efficient and can often solve problems involving tens of thousands of variables and millions of clauses in practice.

First Order Logic (FOL)

FOL aka Predicate Logic

FOL helps us represent knowledge, reason through problems and understand language.

It builds on propositional logic, adding **variables**, **predicates and quantifiers**, making it more expressive and versatile and thus allowing for the expression of more complex relationships and make decisions based on logical reasoning.

Key components of First-Order Logic

1. Constants: These represent specific objects or entities (Example: Alice, 2, NewYork).

2. Variables: These stand for unspecified objects or entities (Example: x, y, z).

3. Predicates: These define properties or relationships (Example: Likes(Alice, Bob) means "Alice likes Bob").

4. Functions: It map objects to other objects (Example: MotherOf(x) refers to the mother of x).

5. Quantifiers: These define the scope of variables:

- **Universal Quantifier (∀)**: Applies a predicate to all elements (Example: ∀x (Person(x) → Mortal(x)) means "All persons are mortal").
- **Existential Quantifier (∃)**: Shows the existence of at least one element (Example: ∃x (Person(x) ∧ Likes(x, IceCream)) means "Someone likes ice cream").
- **Logical Connectives**: Include conjunction (∧), disjunction (∨), implication (→), biconditional (↔) and negation (¬).

Logical Reasoning with FOL

Consider the following statements:

- $\forall$**x (Cat(x)** $\Rightarrow$ **Mammal(x))** (All cats are mammals)
- $\forall$**x (Mammal(x)** $\Rightarrow$ **Animal(x))** (All mammals are animals)
- **Cat(Tom)** (Tom is a cat)

From these, we can logically infer:

- **Mammal(Tom)** (Tom is a mammal)
- **Animal(Tom)** (Tom is an animal)

This example shows how FOL allows machines to derive new knowledge from existing facts through logical reasoning.

Advanced Concepts in FOL

- **Unification**: This involves finding substitutions that make two expressions identical. It's used in automated reasoning to match patterns.

  Example: $P(x, Dog)$ and $P(Alex, y)$. To unify them, we must find substitutions such that both predicates become the same. The solution is $x = Alex$ and $y = Dog$.

- **Resolution**: A rule of inference used in theorem proving to derive contradictions and prove or disprove statements.
- **Model Checking**: This process verifies whether a system meets a specification, used in software and hardware verification.
- **Logic Programming**: It is used in languages like Prolog for AI applications in areas like NLP and expert systems.

Example of Resolution in FOL

Every student at UniTS is smart.

Paolo is a student of UniTS.

Prove that Paolo is smart.

Step 1: Convert statements to First-Order Logic (FOL)

Statement 1: $\forall x \ (Student(x, UniTS) \Rightarrow Smart(x))$

Statement 2: $Student(Paolo, UniTS)$

Statement to Prove: $Smart(Paolo)$

Step 2: Convert FOL sentences into Clausal Form (CNF)

Remove implications:

Clause 1: $\forall x \ \neg Student(x, UniTS) \vee Smart(x)$

Clause 2: $Student(Paolo, UniTS)$

Example of Resolution in FOL

Step 3: Negate the Assertion to be Verified

To apply proof by contradiction, we assume the negation of the goal statement and add it to the set of clauses.

Negated Goal: $\neg Smart(Paolo)$ (Clause 3)

Step 4: Apply Unification

Unify: $x = Paolo$ in Clause 1

Substituting in Clause 1: $\neg Student(Paolo, UniTS) \vee Smart(Paolo)$

Step 5: Use the Resolution Rule

Resolution is applied to eliminate complementary literals and derive a new clause.

Resolving Clause 2 ($Student(Paolo, UniTS)$) with Clause 1 ($\neg Student(Paolo, UniTS) \vee Smart(Paolo)$)

$Student(Paolo, UniTS)$ and $\neg Student(Paolo, UniTS)$ cancel out.

New Clause: $Smart(Paolo)$

Step 6: Repeat Until a Contradiction or Proof is Found

The final resolution step confirms the goal by contradiction. Resolving Clause 3 ($\neg Smart(Paolo)$) with $Smart(Paolo)$. The literals cancel out, resulting in an empty clause ({}).

Exercise on FOL Resolution

Given the following facts:

1.John likes all kinds of food.
2.Apples and vegetables are food.
3.Anything anyone eats and isn't killed by is food.
4.Anil eats peanuts and is still alive.
5.Harry eats everything that Anil eats.
Prove by resolution that: **John likes peanuts**.

Steps by step (proof by contradiction):
- **Step 1:** Convert Facts into First-Order Logic (FOL) Statements
- **Step 2:** Convert FOL Statements into CNF
- **Step 3:** Negate the Statement to be Proven
- **Step 4:** Draw Resolution Graph and Apply Unification

Resolution: $(A \vee B) \wedge (\neg B \vee C)$ we add clause $A \vee C$ until reach an empty set (representing a contradiction)

Propositional Logic VS First-Order Logic

Propositional Logic	First-Order Logic
Represents entire statements as true or false (e.g "It is raining")	Represents relationships, properties and generalizations (e.g "All cats are mammals")
No quantifiers	Uses quantifiers (∀ for "all", ∃ for "some") to express generalizations and existence
Basic logical operations (AND, OR, NOT)	Advanced reasoning through unification, resolution and inference rules
Simple tasks like decision-making and circuit design	Complex tasks like knowledge representation, reasoning and language processing

Modal Logic

Modal Logic

Modal logic is a kind of logic used to represent statements about **necessity** and **possibility**, represented by two unary operators $\Box$ and $\Diamond$ respectively.

For instance the modal formula $\Diamond P$ can be read as "possibly P" while $\Box P$ can be read as "necessarily P". Formulas are assigned truth values relative to a possible world.

In systems of modal logic where $\Box$ and $\Diamond$ are *duals*, $\Box \phi$ can be taken as an abbreviation for $\neg \Diamond \neg \phi$ thus eliminating the need for a separate syntactic rule to introduce it.

However, separate syntactic rules are necessary in systems where the two operators are not interdefinable.

Modal predicate logic is one widely used variant which includes formulas such as $\forall x \ \Diamond P(x)$.

Signal Temporal Logic

[Differentiable STL Monitor]

Linear-time temporal logic which expresses properties over dense time intervals

Syntax of STL

$f(\mathbf{x}) \sim y$ $\sim \in \{ \leq, <, >, \geq, =, \neq \}$	f is a function over the signal x
$\neg\varphi$	Negation
$\varphi \wedge \psi$	Conjuction
$F_{[a,b]}\varphi$	Eventually, at some **Future** step in [a, b]
$G_{[a,b]}\varphi$	**Globally** in all times in [a, b]
$\phi U_{[a,b]}\psi$	For all steps **Until** [a, b]

Atomic formulae are signal predicates over real values

The temperature of the room τ will never be below 20 degrees until it will reach 25 degrees in the next 30 minutes

$$(\tau \geq 20) U_{[0,30]} (\tau \geq 25)$$

Temporal dimension
(truth value is interpreted over time)

Signal Temporal Logic

[Differentiable STL Monitor]

Linear-time temporal logic which expresses properties over dense time intervals

Syntax of STL

$f(\mathbf{x}) \sim y$ $\sim \in \{ \leq, <, >, \geq, =, \neq \}$	f is a function over the signal x
$\neg\varphi$	Negation
$\varphi \wedge \psi$	Conjuction
$F_{[a,b]}\varphi$	Eventually, at some **Future** step in [a, b]
$G_{[a,b]}\varphi$	**Globally** in all times in [a, b]
$\phi U_{[a,b]}\psi$	For all steps **Until** [a, b]

Atomic formulae are signal predicates over real values

The temperature of the room τ will never be below 20 degrees until it will reach 25 degrees in the next 30 minutes

$$(\tau \geq 20) U_{[0,30]} (\tau \geq 25)$$

Temporal dimension
(truth value is interpreted over time)

Signal Temporal Logic

[Differentiable STL Monitor]

Linear-time temporal logic which expresses properties over dense time intervals

Syntax of STL

$f(\mathbf{x}) \sim y$ $\sim \in \{ \leq, <, >, \geq, =, \neq \}$	f is a function over the signal x
$\neg\varphi$	Negation
$\varphi \wedge \psi$	Conjuction
$F_{[a,b]}\varphi$	Eventually, at some **Future** step in [a, b]
$G_{[a,b]}\varphi$	**Globally** in all times in [a, b]
$\phi U_{[a,b]}\psi$	For all steps **Until** [a, b]

Atomic formulae are signal predicates over real values

The temperature of the room τ will reach 25 degrees in the next 60 minutes

$F_{[0,60]}(\tau \geq 25)$

◊ □

Temporal dimension
(truth value is interpreted over time)

Signal Temporal Logic

[Differentiable STL Monitor]

Linear-time temporal logic which expresses properties over dense time intervals

Syntax of STL

$f(\mathbf{x}) \sim y$ $\sim \in \{ \leq, <, >, \geq, =, \neq \}$	f is a function over the signal x
$\neg\varphi$	Negation
$\varphi \wedge \psi$	Conjuction
$\Diamond$ $F_{[a,b]}\varphi$	Eventually, at some **Future** step in [a, b]
$\Box$ $G_{[a,b]}\varphi$	**Globally** in all times in [a, b]
$\phi U_{[a,b]}\psi$	For all steps **Until** [a, b]

Atomic formulae are signal predicates over real values

The temperature of the room τ will never be below 20 degrees in the next 30 minutes

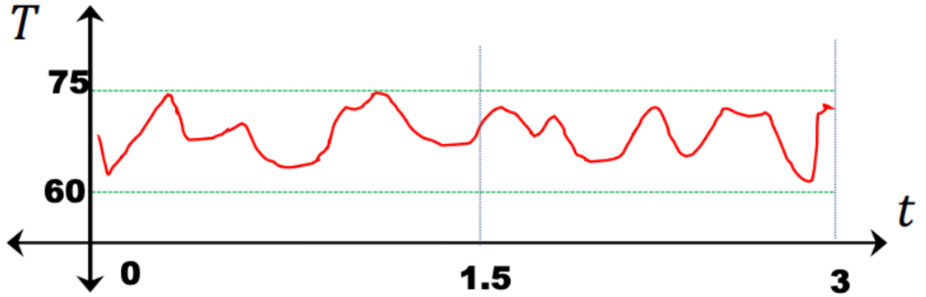
$$G_{[0,30]}(\tau \geq 20) \leftrightarrow \neg(F_{[0,30]}(\tau \leq 20))$$

Temporal dimension
(truth value is interpreted over time)

STL Specifications

$\mathbf{G}_{[0,3]}\ (60 < T < 75)$

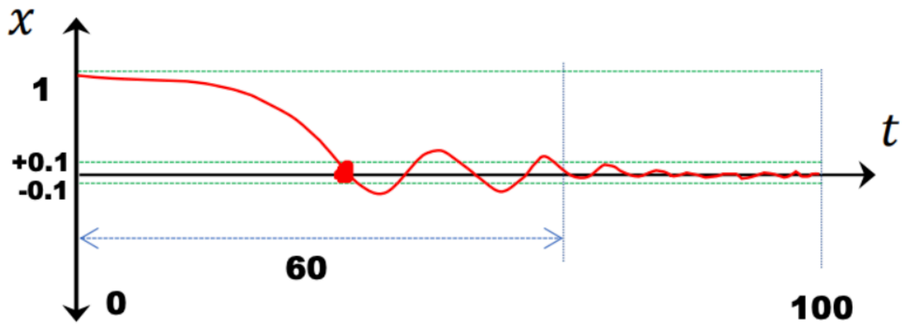
Always between time 0 and 3



$\mathbf{F}_{[0,60]}(\mathbf{G}\ (|x| < 0.1)\ )$

Eventually at **some time** *t* between time 0 and 60

From that time *t*, always till the end of the signal trace



STL Syntax

$$\phi := true \mid \mu \mid \neg\phi \mid \phi \wedge \phi \mid \phi \; U_I \; \phi,$$

STL Boolean Semantics

$$\begin{aligned}
(\vec{s}, t) \models \mu &\Leftrightarrow g(\vec{s}(t)) > 0 \\
(\vec{s}, t) \models \neg\phi &\Leftrightarrow \neg((\vec{s}, t) \models \phi) \\
(\vec{s}, t) \models \phi_1 \wedge \phi_2 &\Leftrightarrow (\vec{s}, t) \models \phi_1 \wedge (\vec{s}, t) \models \phi_2 \\
(\vec{s}, t) \models \phi_1 U_{[a,b]} \phi_2 &\Leftrightarrow \exists\, t' \in [t+a, t+b] \text{ s.t. } (\vec{s}, t') \models \phi_2 \; \forall\, t'' \in [t, t'],\; (\vec{s}, t'') \models \phi_1
\end{aligned}$$

STL Quantitative Semantics

$$\begin{aligned}
\mathtt{Rob}_{\mu}(\vec{s}, t) &= g(\vec{s}(t)) \\
\mathtt{Rob}_{\neg\phi}(\vec{s}, t) &= -\mathtt{Rob}_{\phi}(\vec{s}, t) \\
\mathtt{Rob}_{\phi_1 \wedge \phi_2}(\vec{s}, t) &= \min(\mathtt{Rob}_{\phi_1}(\vec{s}, t), \mathtt{Rob}_{\phi_2}(\vec{s}, t)) \\
\mathtt{Rob}_{\phi_1 U_{[a,b]} \phi_2}(\vec{s}, t) &= \sup_{t' \in [t+a, t+b]} \left(\min \left(\mathtt{Rob}_{\phi_2}(\vec{s}, t'), \inf_{t'' \in [t, t']} \mathtt{Rob}_{\phi_1}(\vec{s}, t'') \right) \right)
\end{aligned}$$

Derived operators

$$false := \neg true,\ \varphi \vee \psi := \neg(\neg\varphi \wedge \neg\psi),$$

$$F_{[a,b]}\varphi := true\ U_{[a,b]}\varphi,\ G_{[a,b]}\varphi := \neg F_{[a,b]}\neg\varphi$$

STL Boolean Semantics

Eventually: $(\vec{s}, t) \models F_{[a,b]}\varphi \iff \exists t' \in [t+a, t+b]$ s.t. $(\vec{s}, t') \models \varphi$;

Always: $(\vec{s}, t) \models G_{[a,b]}\varphi \iff \forall t' \in [t+a, t+b] \quad (\vec{s}, t') \models \varphi$.

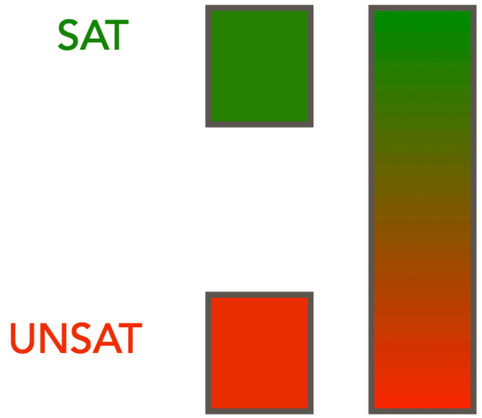
STL Quantitative Semantics

Eventually: $\mathtt{Rob}_{F_{[a,b]}\varphi}(\vec{s}, t) = \sup_{t' \in [t+a, t+b]} \left(\mathtt{Rob}_{\varphi}(\vec{s}, t')\right)$

Always: $\mathtt{Rob}_{G_{[a,b]}\varphi}(\vec{s}, t) = \inf_{t' \in [t+a, t+b]} \left(\mathtt{Rob}_{\varphi}(\vec{s}, t')\right)$

Signal Temporal Logic

Semantics (interpreted over a signal/trajectory)



Both **Qualitative** (Boolean) and **Quantitative** (Robust) **semantics**

Soundness

sign of the robustness indicates Boolean satisfaction

$$\rho(\varphi, \xi, t) > 0 \rightarrow s(\varphi, \xi, t) = 1$$

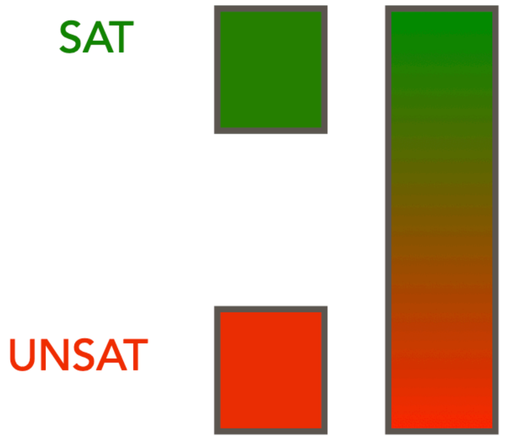
$$\rho(\varphi, \xi, t) < 0 \rightarrow s(\varphi, \xi, t) = 0$$

$\rho(\varphi, \xi, t) \in \mathbb{R}$ indicating "*how much*" the STL formula φ is (un)satisfied for the trajectory ξ at time t

$s(\varphi, \xi, t) \in \{0,1\}$ indicating whether the STL formula φ is true for the trajectory ξ at time t

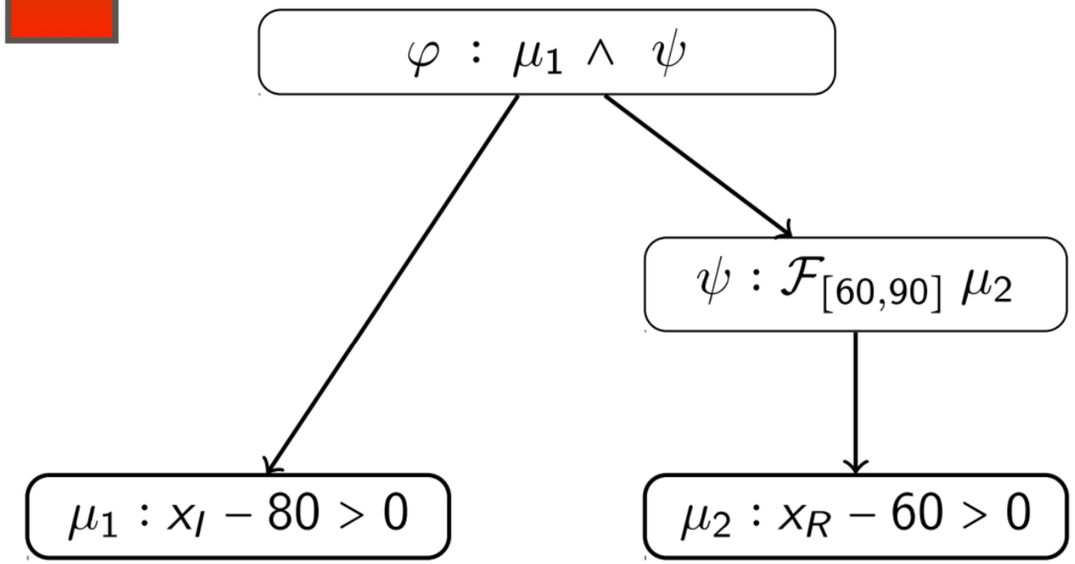
Signal Temporal Logic

Semantics (interpreted over a signal/trajectory)



Both **Qualitative** (Boolean) and **Quantitative** (Robust) **semantics**

$$\varphi : (x_I > 80) \wedge \mathcal{F}_{[60,90]}\,(x_R > 60)$$



Soundness

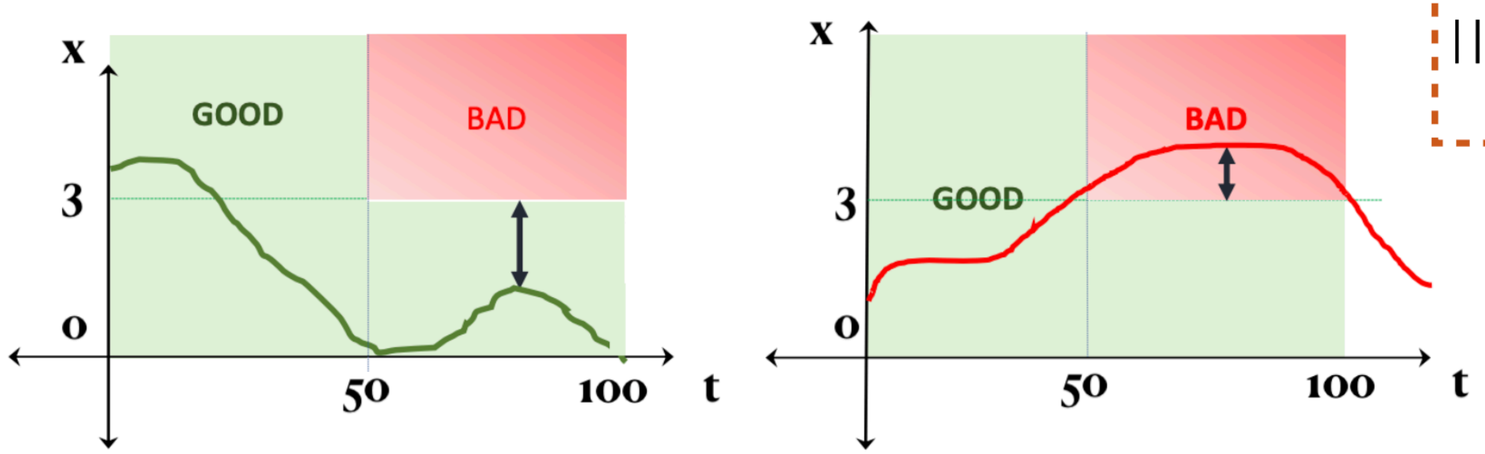
sign of the robustness indicates Boolean satisfaction

$$\rho(\varphi, \xi, t) > 0 \rightarrow s(\varphi, \xi, t) = 1$$
$$\rho(\varphi, \xi, t) < 0 \rightarrow s(\varphi, \xi, t) = 0$$

Signal Temporal Logic

Robustness encodes distance to violation/satisfaction
(sign indicates satisfaction status)

$$G_{[50,100]}(x < 3)$$



Correctness

the value of the robustness indicates resilience to perturbations

$$||\xi_1 - \xi_2||_\infty < \rho(\varphi, \xi_1, t) \rightarrow s(\varphi, \xi_1, t) = s(\varphi, \xi_2, t)$$

Logic Programming

Logic based reasoning

Logic as a formal system comprised of **syntax**, **semantics** and **inference rules**.

Knowledge and computational problems are expressed using logical facts and rules.

Different logics may suit different applications.

1. Logic Programming,
2. Probabilistic Logic Programming,
3. Fuzzy Logic

used differently to reason on graph structures.

1. Logic Programming

head body

Formulas of the form $H \leftarrow B_1 \wedge \cdots \wedge B_n$

"H is true if all the conditions $B_1, \ldots, B_n$ hold."

Interpreted as **rules** where the body $B_1 \wedge \cdots \wedge B_n$ implies the (positive) head H,

or as **facts**, if the body is empty.

Example

$likes(alice, bob)$

$likes(bob, alice)$

background knowledge

$friends(X, Y) \leftarrow likes(X, Y) \wedge likes(Y, X)$ rule

```
</> prolog

% Background knowledge (facts)
likes(alice, bob).
likes(bob, alice).

% Rule
friends(X, Y) :- likes(X, Y), likes(Y, X).
```

```
</> prolog

% Example query
?- friends(alice, bob).
```

```
Output: True
```

```
</> prolog

% Example query
?- friends(X, Y).
```

```
Output:
X=alice,Y=bob;
X=bob,Y=alice.
```

1. Logic Programming

head body

Formulas of the form $H \leftarrow B_1 \wedge \cdots \wedge B_n$

"H is true if all the conditions $B_1, \ldots, B_n$ hold."

Interpreted as **rules** where the body $B_1 \wedge \cdots \wedge B_n$ implies the (positive) head H,

or as **facts**, if the body is empty.

Example

$likes(alice, bob)$

$likes(bob, alice)$

background knowledge

$friends(X, Y) \leftarrow likes(X, Y) \wedge likes(Y, X)$ rule

Intuitively, we can deduce that $friends(alice, bob)$.

```
</> prolog

% Background knowledge (facts)
likes(alice, bob).
likes(bob, alice).

% Rule
friends(X, Y) :- likes(X, Y), likes(Y, X).
```

```
</> prolog

% Example query
?- friends(alice, bob).
```

```
Output: True
```

```
</> prolog

% Example query
?- friends(X, Y).
```

```
Output:
X=alice,Y=bob;
X=bob,Y=alice.
```

1. Logic Programming

Different logic programming languages adopt different semantics and inference mechanisms.

Prolog

What can we prove?

Logical inference

ASP

Which solution satisfy the constraints?

Constraint-based reasoning

Problog

How likely is a conclusion?

Probabilistic inference

ILASP

Which rules explain the examples?

Inductive rule learning

These approaches share a logic-based representation, but differ in their objectives, semantics, and computational mechanisms.

1. **Prolog:** Query-Driven Logical Inference

Prolog (Programming in Logic) is a logic programming language primarily based on Horn clauses and goal-directed inference.

```
</> prolog

parent(alice, bob).
parent(bob, charlie).

grandparent(X,Z) :- parent(X,Y), parent(Y,Z).
```

We can ask:

```
</> prolog

?- grandparent(alice, charlie).
```

```
Result: True
```

Or query for all grandparents:

```
</> prolog

?- grandparent(X, charlie).
```

```
Result: X=alice
```

How does Prolog reason?

- Starts from a query (goal).
- Matches goals against facts and rules through unification.
- Uses backward chaining and backtracking to find proofs.

Key idea: Prolog searches for substitutions that make a query succed.

1. Answer Set Programming (ASP)

Answer Set Programming (ASP) is a declarative logic programming paradigm for knowledge representation and combinatorial problem solving.

Instead of proving individual queries, ASP computes answer sets (stable models) representing possible solutions to the entire program.

An ASP program can contain:

- **Facts:** known information.
- **Rules:** logical relationships and deductions.
- **Choice rules:** possible alternatives.
- **Constraints:** conditions that must not be violated.

```
</> asp

bird(tweety).
bird(polly).
penguin(polly).

flies(X) :- bird(X), not penguin(X).
```

The resulting answer set contains:

```
bird(tweety)
bird(polly)
penguin(polly)
flies(tweety)
```

Here, `not penguin(X)` uses default negation (negation as failure): we assume that `penguin(X)` does not hold unless it can be established.

This enables non-monotonic reasoning: adding new facts may invalidate previously derived conclusions.

1. **ASP:** Choice rules and Constraints

Consider two robots, `r1` and `r2`, and two tasks, `t1` and `t2`.

We want to assign each task to exactly one robot, but `r1` cannot perform `t2`.

</> asp

```
robot(r1; r2).
task(t1; t2).

% Exactly one robot per task
1 { assign(R,T) : robot(R) } 1 :- task(T).

% Robot r1 cannot execute task t2
:- assign(r1,t2).

#show assign/2.
```

The solver finds two answer sets:

Answer set 1	**Answer set 2**
`r1 → t1`	`r2 → t1`
`r2 → t2`	`R2 → t2`

Both solutions satisfy all constraints. The program does not require different tasks to be assigned to different robots.

Solvers such as **Clingo** typically ground the program and compute its stable models.

1. ILASP: Learning Logic Programs from Examples

ILASP (Inductive Learning of Answer Set Programs) is an Inductive Logic Programming (ILP) system that automatically learns ASP rules from examples.

In traditional ASP: Facts + Rules $\xrightarrow{\text{ASP}}$ Answer Sets

In ILASP: Examples + Background Knowledge $\xrightarrow{\text{ILASP}}$ Learned Rules

An ILASP learning task consists of:

Background knowledge (B): what is already known.

Examples (E): desired and undesired behaviors.

Hypothesis space (S): which rules are allowed to be learned.

The **goal** is to find a hypothesis $H \subseteq S$ such that $B \cup H$ explains the examples under ASP semantics.

Key difference: ASP applies rules, whereas ILASP learns rules.

1. ILASP Example: Learning When a Robot Can Enter a Room

Suppose we observe the following situations:

Robot	Charged	Door open	Can enter
r1	Yes	Yes	Yes
r2	Yes	No	No
r3	No	Yes	No

We want ILASP to discover the conditions under which a robot can enter.

</> ilasp

```
% Allowed rule structure
#modeh(can_enter(var(robot))).
#modeb(charged(var(robot))).
#modeb(door_open(var(robot))).
#maxv(1).

% Positive example
#pos({can_enter(r1)}, {}, {
    robot(r1).
    charged(r1).
    door_open(r1).
}).
```

</> ilasp

```
% Negative examples
#neg({can_enter(r2)}, {}, {
    robot(r2).
    charged(r2).
}).

#neg({can_enter(r3)}, {}, {
    robot(r3).
    door_open(r3).
}).
```

A possible learned rule is:

</> ilasp

```
can_enter(X) :-
charged(X), door_open(X).
```

1. ILASP

```
% Allowed rule structure
#modeh(can_enter(var(robot))).
#modeb(charged(var(robot))).
#modeb(door_open(var(robot)),
(positive)).
#maxv(1).

% Positive example
#pos({inclusions},
{exclusions}, {context}).

% Negative example
#neg({inclusions},
{exclusions}, {context}).
```

Define family of rules

Which predicates can appear in the **h**ead/**b**ody of the rule

`(positive)`: Predicate allowed only in positive form (without not)

`maxv(1)`: Every rule can containt at most one distinct variable

Define examples

`inclusions`: atoms that must be present in the answer set.

`exclusions`: atoms that should not be in the answer set.

`context`: facts and rules valid only for tha example

1. `#pos(I,E,C)` requires that at least one answer set exists that is compatible with the specified inclusions and exclusions.

2. `#neg(I,E,C)` requires that no answer set exists that is compatible with the specified inclusions and exclusions.

`#pos({can_enter(r1)}, {}, {robot(r1). charged(r1). door_open(r1).}).`
in this context, we require at least one answer set containing `can_enter(r1)`

`#neg({can_enter(r2)}, {}, C)`
excludes any answer set containing `can_enter(r2)` in context C. It is not equivalent to simply asserting a negative fact.

How is the rule selected?

`can_enter(X) :- charged(X).` is not valid: it would allow `r2` to enter.

`can_enter(X) :- door_open(X).` is not valid: it would allow `r3` to enter.

`can_enter(X) :- charged(X), door_open(X).` correctly explains all three examples.

1. ILASP

ILASP can learn:

Logical rules: relationships beween predicates

```
can_enter(X) :- charged(X), door_open(X).
```

Constraints: configurations that must be forbidden

```
:- assign(R,T), unavailable(R).
```

Preferences (weak constraints): learn which valid configuration should be preferred

```
:~ assign(R,T), costly(R,T). [1@1,R,T]
```

Limitation: Learning preferences requires ordering examples, which express that one solution should be preferred to another.

2. Probabilistic Logic Programming

Probabilistic logic with **soft rules**, where a probability $p \in [0,1]$ is attached. Many tools (e.g. `ProbLog`) take this probabilistic approach as it allows noisy data and uncertainty, and takes advantage of statistical methods.

Example: $0.75 :: car(X) \leftarrow wheels(X,4) \wedge drivable(X)$

says that we are 75% certain that four-wheeled drivable objects are cars.

In `ProbLog` all the rules are soft and independent and the resolution algorithm is extended to keep track of probabilities.

2. **ProbLog:** Probabilistic Logic Programming

ProbLog extends Prolog-style logic programming with probabilistic facts.

Instead of treating all facts as certainly true, we can attach probabilities to them.

```
</> prolog

0.8::rain.
0.6::sprinkler.

wet_grass :- rain.
wet_grass :- sprinkler.

query(wet_grass)
```

This program represents uncertainty about rain and sprinkler activation.

We want to compute: $P(\text{wet_grass})$

Assuming the two probabilistic facts are independent:

$$P(\text{wet_grass}) = P(\text{rain} \vee \text{sprinkler})$$
$$= 1 - (1 - 0.8)(1 - 0.6) = 0.92$$

Key idea: ProbLog performs probabilistic inference over possible worlds generated by probabilistic facts.

2. **ProbLog:** Conditioning on Evidence

ProbLog can also incorporate observations to update probabilities.

Example:

```
</> prolog

0.2::disease.

0.9::positive_test :- disease.
0.1::positive_test :- \
+disease.

evidence(positive_test, true).
query(disease).
```

We know:

$$P(D) = 0.2$$
$$P(+ \mid D) = 0.9$$
$$P(+ \mid \neg D) = 0.1$$

Using Bayes' theorem:

$$P(D \mid +) = \frac{P(+ \mid D)P(D)}{P(+)} = \frac{0.9 \cdot 0.2}{0.9 \cdot 0.2 + 0.1 \cdot 0.8} \approx 0.692$$

So, after observing a positive test, the probability of disease increases from 20% to approximately 69.2%.

Comparing Prolog, ASP, ILASP and ProbLog

	Prolog	ASP	ILASP	ProbLog
Primary task	Prove goals	Find stable models	Learn rules	Infer probabilities
Inputs	Fact, rules, query	Fact, rules, constraints	Background, examples, hypothesis space	Probabilistic facts, rules, evidence
Output	Query answers	Answer sets	ASP program	Query probabilities
Main paradigm	Deductive	Declarative/non-monotonic	Inductive	Probabilistic
Uncertainty	Non native	Logical alternatives and defaults	Learning from examples	Quantitative probabilities
Typical application	Expert systems	Planning and scheduling	Rule discovery	Reason under uncertainty

Prolog / ASP
Apply symbolic knowledge to reason

ILASP
Learn symbolic knowledge from examples

Problog
Represent and propagate probabilistic uncertainty

2. Probabilistic Logic Programming

Alternatively, we can attach a weight w_i to every first-order formula F_i in a knowledge base, i.e. $(F_i, w_i)_{i=1}^n$. Assuming a language with finitely many constants, define a graph called a **Markov Logic Network** (MLN).

An *atom* is a formula composed of constants, variables and functions but with no logical connectives. A formula is *grounded* if it has no variables.

The **nodes** of a MLN are every grounding of every atom appearing in the F_i, and there is an **edge** if the ground atoms appear together in at least one of the F_i.

2. Probabilistic Logic Programming

Example of Markov Logic Network

Constants A and B and the following formulas with weights 1.3 and 0.7 respectively

$$\forall x \,.\, Vegan(x) \Rightarrow Healthy(x)$$

$$\forall x \,.\, \forall y \,.\, Friends(x, y) \Rightarrow (Vegan(x) \iff Vegan(y))$$

2. Probabilistic Logic Programming

Example of Markov Logic Network

w_i

Constants A and B and the following formulas with weights 1.3 and 0.7 respectively

F_i

$$\forall x \,.\, Vegan(x) \Rightarrow Healthy(x)$$

$$\forall x \,.\, \forall y \,.\, Friends(x, y) \Rightarrow (Vegan(x) \iff Vegan(y))$$

2. Probabilistic Logic Programming

Example of Markov Logic Network

w_i

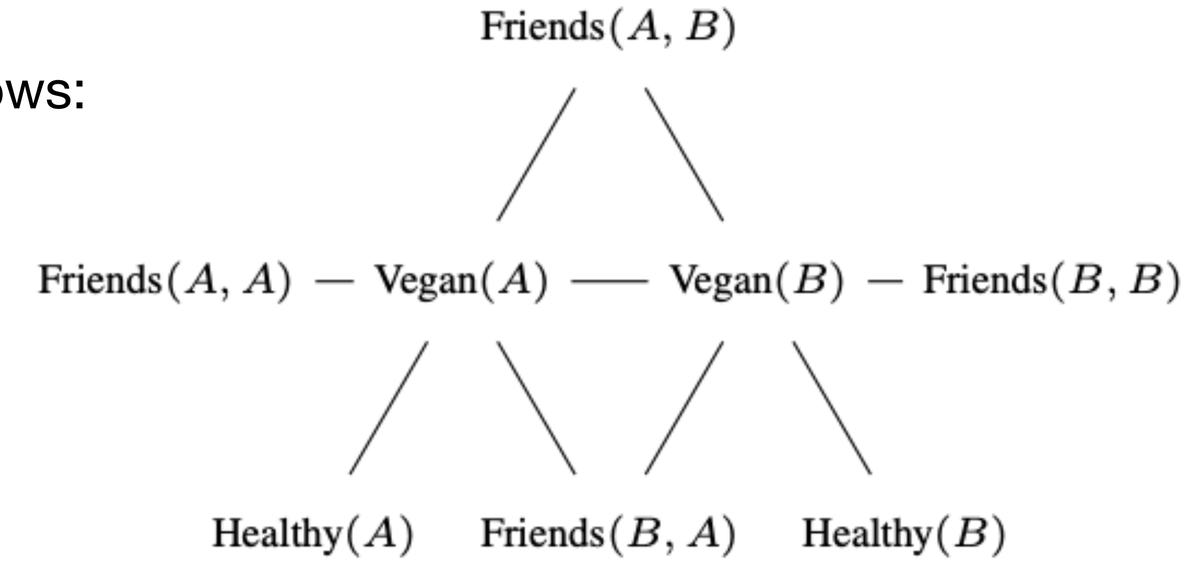
Constants A and B and the following formulas with weights 1.3 and 0.7 respectively

F_i

$$\forall x \,.\, Vegan(x) \Rightarrow Healthy(x)$$

$$\forall x \,.\, \forall y \,.\, Friends(x, y) \Rightarrow (Vegan(x) \iff Vegan(y))$$

The ground Markov network looks as follows:



2. Markov Logic Networks

A possible *world*, x, is a truth value assignment to each node.

The probability distribution over the possible worlds, denoted by the random variable X, is given by

$$P(X = x) = \frac{\exp(\sum_{i=1}^{n} w_i n_i(x))}{\sum_y \exp(\sum_{i=1}^{n} w_i n_i(y))}$$

softmax probability among all possible worlds

where n is the number of formulas and $n_i(x)$ is the number of true groundings of F_i in x.

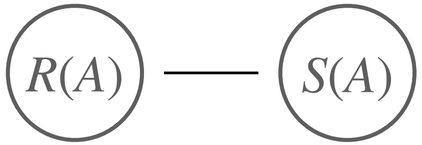
Example: Let x be the world where A and B are vegan, unhealthy and have no friends.

Exercise 1: compute $P(X = x)$

Markov Logic Networks

Simple example

$\forall x \,.\, R(x) \Rightarrow S(x)$ with weight w and constant $C = \{A\}$



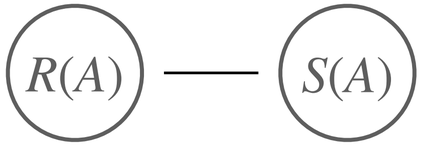
4 possible worlds

$\{\neg R(A), \neg S(A)\}$ $\{\neg R(A), S(A)\}$ $\{R(A), \neg S(A)\}$ $\{R(A), S(A)\}$

Markov Logic Networks

Simple example

$\forall x \,.\, R(x) \Rightarrow S(x)$ with weight w and constant $C = \{A\}$



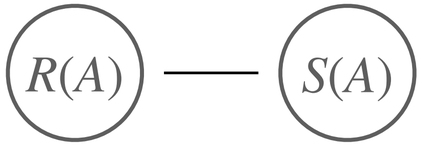
4 possible worlds

$\{\neg R(A), \neg S(A)\}$	$\{\neg R(A), S(A)\}$	$\{R(A), \neg S(A)\}$	$\{R(A), S(A)\}$
1	1	0	1

Markov Logic Networks

Simple example

$\forall x \,.\, R(x) \Rightarrow S(x)$ with weight w and constant $C = \{A\}$



4 possible worlds

$\{\neg R(A), \neg S(A)\}$	$\{\neg R(A), S(A)\}$	$\{R(A), \neg S(A)\}$	$\{R(A), S(A)\}$
1	1	0	1

$$P(\{R(A), \neg S(A)\}) = \frac{1}{3e^w + 1}$$

2. Probabilistic Logic Programming

Table I. Example of a first-order knowledge base and MLN. $\texttt{Fr}()$ is short for $\texttt{Friends}()$, $\texttt{Sm}()$ for $\texttt{Smokes}()$, and $\texttt{Ca}()$ for $\texttt{Cancer}()$.

English	First-Order Logic	Clausal Form	Weight
Friends of friends are friends.	$\forall \mathtt{x} \forall \mathtt{y} \forall \mathtt{z}\ \mathtt{Fr}(\mathtt{x},\mathtt{y}) \wedge \mathtt{Fr}(\mathtt{y},\mathtt{z}) \Rightarrow \mathtt{Fr}(\mathtt{x},\mathtt{z})$	$\neg\mathtt{Fr}(\mathtt{x},\mathtt{y}) \vee \neg\mathtt{Fr}(\mathtt{y},\mathtt{z}) \vee \mathtt{Fr}(\mathtt{x},\mathtt{z})$	0.7
Friendless people smoke.	$\forall \mathtt{x}\ (\neg(\exists \mathtt{y}\ \mathtt{Fr}(\mathtt{x},\mathtt{y})) \Rightarrow \mathtt{Sm}(\mathtt{x}))$	$\mathtt{Fr}(\mathtt{x},\mathtt{g}(\mathtt{x})) \vee \mathtt{Sm}(\mathtt{x})$	2.3
Smoking causes cancer.	$\forall \mathtt{x}\ \mathtt{Sm}(\mathtt{x}) \Rightarrow \mathtt{Ca}(\mathtt{x})$	$\neg\mathtt{Sm}(\mathtt{x}) \vee \mathtt{Ca}(\mathtt{x})$	1.5
If two people are friends, either	$\forall \mathtt{x} \forall \mathtt{y}\ \mathtt{Fr}(\mathtt{x},\mathtt{y}) \Rightarrow (\mathtt{Sm}(\mathtt{x}) \Leftrightarrow \mathtt{Sm}(\mathtt{y}))$	$\neg\mathtt{Fr}(\mathtt{x},\mathtt{y}) \vee \mathtt{Sm}(\mathtt{x}) \vee \neg\mathtt{Sm}(\mathtt{y})$,	1.1
both smoke or neither does.		$\neg\mathtt{Fr}(\mathtt{x},\mathtt{y}) \vee \neg\mathtt{Sm}(\mathtt{x}) \vee \mathtt{Sm}(\mathtt{y})$	1.1

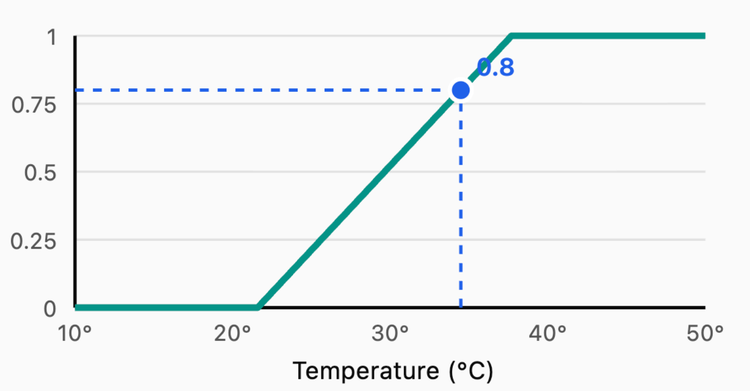
3. **Fuzzy Logic:** Reasoning with degrees of truth

Instead of associating a weight (or probability) to each formula, representing the “degree of belief”, we can also attach a value to the “**degree of truth**” of the statement.

Fuzzy Logic generalizes classical Boolean logic by allowing propositions to have truth values anywhere in the interval $[0,1]$.

In classical logic: $v(P) \in \{0,1\}$

In fuzzy logic: $v(P) \in [0,1]$



Example: Degree of truth of "The room is hot"

The interpretation of the usual boolean logical connectives is adjusted. For instance, a fuzzy version of $x \wedge y$ could be $\min(x, y)$ where x and y are in the range $[0,1]$.

In general, any binary operation on [0, 1] that is commutative, monotone, associative, and respects the identity is called a *t-norm* and can represent conjunction in fuzzy logic.

3. Fuzzy Logic: Reasoning with degrees of truth

One common choice of fuzzy connectives is:

Operator	Definition
$\neg A$	$1 - v(A)$
$A \wedge B$	$\min(v(A), v(B))$
$A \vee B$	$\max(v(A), v(B))$

Other fuzzy logics, such as Łukasiewicz and product logic, use different operators.

Why is fuzzy logic relevant for Neuro-Symbolic AI?

- It provides continuous relaxations of logical expressions.
- Certain fuzzy operators are differentiable or differentiable almost everywhere.
- It allows logical constraints to contribute to optimization objectives.
- It is used in neuro-symbolic approaches such as Logic Tensor Networks

	Fuzzy Logic	**Probabilistic Logic**
Value in [0,1]	Degree of truth	Probability
Main concern	Vagueness	Uncertainty
Typical operations	Fuzzy connectives	Probability calculus
NeSy relevance	Differentiable logical constraints	Uncertainty-aware inference

Bibliography

- https://www.logicmatters.net/resources/pdfs/IFL2_LM.pdf
- https://www.geeksforgeeks.org/engineering-mathematics/proposition-logic/
- http://intrologic.stanford.edu/chapters/chapter_01.html
- DeLong, L. N., Mir, R. F., & Fleuriot, J. D. (**2024**). **Neurosymbolic AI for reasoning over knowledge graphs: A survey**. *IEEE Transactions on Neural Networks and Learning Systems*

Coming next…

Logic Tensor Networks